Neural Conjecture Generation for Automated Theorem Proving via Symbol-Anonymous Graph Neural Networks

Anonymous Authors
anonymous@example.com

March 20, 2026

Abstract

We present a neural system for generating useful intermediate lemmas ("cuts") that speed up automated theorem provers. Given a first-order logic problem in clausal normal form (CNF), our system encodes it as a heterogeneous graph using a graph neural network with an optional *named symbol embedding* layer—where well-known Mizar library symbols receive learnable embeddings while problem-specific Skolem symbols remain fully anonymous. A pointer-based autoregressive decoder then generates candidate lemmas by selecting symbols from the encoded graph. We introduce *arity-constrained decoding*, a generation-time technique that enforces syntactic validity without retraining, improving validity from 2% to 85–88%. We systematically compare eight model configurations across five decoder architectures (Transformer, SSM/Mamba-style, conditional VAE, graph growing, and subgraph completion) on the Mizar40 dataset of 3,161 problems with 122,356 labeled cuts, using a proper train/validation/test split of 2,239/279/279 problems. Named symbol embeddings reduce validation loss from 0.474 to 0.289 (best configuration). On 279 held-out test problems, the best model generates conjectures of which 61% are provably correct lemmas and 5.8% provide actual proof speedups, with the best achieving a **77× reduction** in proof search. These results demonstrate that combining structural graph representations with targeted symbol semantics yields effective conjecturing strategies that generalize to unseen mathematical theories.

1 Introduction

Automated theorem provers (ATPs) such as E [12], Vampire [9], and SPASS [16] search for proofs by systematically exploring the space of logical inferences. For hard problems, this search can require processing thousands or millions of clauses. A well-chosen intermediate lemma—a *cut*—can dramatically reduce this search by decomposing a hard problem into two easier subproblems.

Finding good cuts traditionally requires either human mathematical insight or expensive enumeration. Recent work has explored using machine learning to guide proof search [7, 10, 2], but *generating* novel intermediate lemmas remains largely unexplored.

We address this challenge with a neural system that:

1. Encodes CNF problems as heterogeneous graphs with a **GNN encoder** supporting both fully symbol-anonymous and hybrid named-embedding modes;
2. Introduces **named symbol embeddings** for $\sim$3,097 Mizar library symbols while keeping Skolem symbols anonymous, reducing validation loss from 0.474 to 0.289;
3. Generates candidate lemmas via a **pointer-based autoregressive decoder** that selects symbols directly from the encoded graph;
4. Enforces syntactic validity through **arity-constrained decoding**, a generation-time state machine requiring no retraining;
5. Achieves **61% provability rate** on test-set conjectures, **5.8% useful speedup rate**, and up to **77× proof speedup**.

Our key finding is that combining structural anonymity with targeted name-based priors yields the best results: the model achieves 98% symbol precision in anonymous mode (almost never generating symbols absent from the problem), and named embeddings further improve the quality of generated conjectures across all three competitive decoder architectures. Surprisingly, validation loss does not predict test-set usefulness: the model with the worst validation loss among named configurations (D+named, 0.413) achieves the best test useful rate (5.8%), revealing a fundamental exploration–exploitation trade-off in conjecture generation.

2 Related Work

Machine learning for theorem proving. ENIGMA [7] uses gradient-boosted trees to select given clauses in saturation-based provers. DeepMath [1] and FormulaNet [15] learn premise selection for theorem proving. HOList [2] applies reinforcement learning to interactive theorem proving. GraphSAGE-based approaches [11] encode formulas as graphs for premise selection. Our work differs in *generating* novel lemmas rather than selecting from existing ones.

Conjecture generation. TerpreT [4] and related work explore conjecture synthesis in interactive provers. The INT system [17] generates synthetic theorems for curriculum learning. Most prior work operates on higher-level representations; we work directly with first-order CNF clauses at the prover level.

Graph neural networks for logic. GNNs have been applied to SAT solving [13], formula classification [3], and premise selection [11]. Our heterogeneous graph representation with 5 node types and 16 edge types is specifically designed for the rich structure of first-order CNF clauses.

Pointer networks. Pointer networks [14] select from variable-size input vocabularies, which is essential when the output must reference symbols from the specific input problem. Our decoder extends this with structured tree generation.

3 Problem Formulation

3.1 CNF Problems and Cuts

A CNF problem P consists of clauses $C_1, \ldots, C_n$, where each clause $C_i = l_1 \vee l_2 \vee \cdots \vee l_m$ is a disjunction of literals. Each literal is a possibly-negated predicate applied to terms, which may be variables (universally quantified), constants, or function applications.

Given a candidate lemma C (itself a CNF clause), we define the *cut evaluation*:

- **P1**: Prove P with C added as an axiom (does C help?)
- **P2**: Prove C from P by refuting $P \cup \{\neg C\}$ (is C valid?)

If both succeed with proof search lengths L_1 and L_2 respectively, the *speedup ratio* is $r = (L_1 + L_2)/L$, where L is the original proof search length. A ratio $r < 1$ indicates a useful cut.

3.2 Dataset

We use the Mizar40 cut dataset [8], consisting of 3,161 CNF problems with 122,356 evaluated cuts (Table 1). Each cut is a clause extracted from E prover's proof, with a measured speedup ratio.

We split by *problem* (not by cut) to prevent data leakage: 2,239 problems for training (80%), 279 for validation (10%), and 279 for testing (10%). All cuts for a given problem go to the same split. The split is deterministic (seed=42) based on sorted problem names.

Table 1: Dataset statistics.

Metric	Value
Total problems	3,161
Total evaluated cuts	122,356
Good cuts ($r < 1.0$)	44,895 (36.7%)
Excellent cuts ($r < 0.1$)	1,509 (1.2%)
Unique Mizar symbols	5,105
Unique Skolem patterns	305
Median graph nodes per problem	395

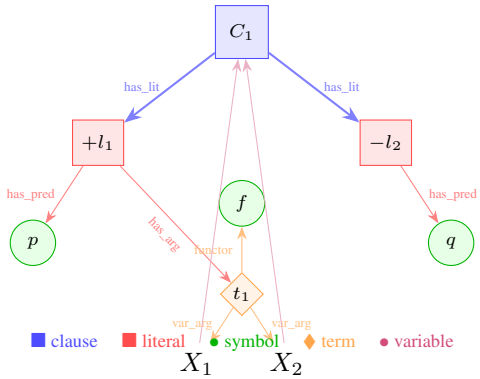

Figure 1: Heterogeneous graph representation of a CNF clause $C_1 = p(f(X_1, X_2)) \vee \neg q(\ldots)$. Symbol nodes (green) are shared across all occurrences—the same symbol node is used wherever a predicate or function appears, enabling the GNN to learn its structural role.

4 Method

4.1 Graph Representation

Each CNF problem is represented as a typed heterogeneous graph with five node types and 16 directed edge types (8 bidirectional pairs). Figure 1 illustrates the structure.

Symbol anonymity (base mode). In the base anonymous mode, symbol nodes receive *only structural features*: a 4-dimensional vector encoding (is_predicate, is_function, is_constant, normalized_arity). No name-based embedding is used. After T rounds of message passing, each symbol's embedding encodes its structural role: arity, co-occurrence patterns, variable flow, and position within clauses. This enables cross-theory transfer: structurally similar symbols in different theories (e.g., group multiplication and ring multiplication) acquire similar embeddings.

4.2 Named Symbol Embeddings (Hybrid Mode)

While anonymous mode achieves strong performance, it treats structurally similar symbols identically—for example, addition (`k2_xcmplx_0`) and multiplication (`k3_xcmplx_0`), both binary functions in arithmetic contexts, receive indistinguishable initial features. We introduce a hybrid mode that provides learnable name-based embeddings for known symbols while preserving full anonymity for problem-specific Skolem symbols.

Vocabulary construction. All Mizar symbols appearing in at least 2 problems are assigned vocabulary indices, yielding ∼3,097 named symbols (plus index 0 for UNK). Skolem symbols (matching pattern `esk\d+_\d+`) always receive the UNK index regardless of frequency.

Table 2: Action types for clause generation. PRED and ARG_FUNC use pointer attention over the encoder's symbol embeddings.

Action	Argument	Description
NEW_LIT_POS	—	Start positive literal
NEW_LIT_NEG	—	Start negative literal
PRED	symbol pointer	Select predicate
ARG_VAR	variable slot	Argument is a variable
ARG_FUNC	symbol pointer	Argument is a function application
END_ARGS	—	Close current arguments
END_CLAUSE	—	Generation complete

Embedding integration. The name embedding is *concatenated* with the 4-dimensional structural features, then projected to the hidden dimension:

$$\mathbf{h}_s^{(0)} = \text{Linear}_{4+d\to d}\big([\mathbf{f}_s \,\|\, \mathbf{e}_{\text{name}(s)}]\big) \tag{1}$$

where $\mathbf{f}_s \in \mathbb{R}^4$ is the structural feature vector and $\mathbf{e}_{\text{name}(s)} \in \mathbb{R}^d$ is the name embedding (zero for Skolem symbols via `padding_idx=0`). This ensures the structural signal is always present; the name embedding provides additional prior information but cannot override it.

Impact. Named embeddings consistently reduce validation loss by 12–38% across all three autoregressive architectures (Table 4), with the improvement largest for the Transformer (38%) and smallest for the SSM (13%). The gain comes from disambiguating structurally similar symbols and providing prior knowledge about mathematical semantics.

4.3 GNN Encoder

The encoder is a heterogeneous GNN with T=6 layers of SAGEConv-based message passing [6]:

$$\mathbf{h}_v^{(t+1)} = \text{ReLU}\left(\text{LN}\left(\mathbf{h}_v^{(t)} + \sum_{r\in\mathcal{R}} \text{SAGEConv}_r^{(t)}\left(\mathbf{h}_v^{(t)}, \{\mathbf{h}_u^{(t)} : (u,r,v)\in\mathcal{E}\}\right)\right)\right) \tag{2}$$

where $\mathcal{R}$ is the set of 16 edge types, LN is layer normalization, and each $\text{SAGEConv}_r^{(t)}$ has independent parameters. Input features are projected to the shared hidden dimension d=128 via per-type linear layers.

4.4 Decoder: Pointer-Based Autoregressive Generation

Target clauses are encoded as action sequences (Table 2) that the decoder generates autoregressively.

The **Transformer decoder** (our primary architecture) consists of 3 TransformerDecoderLayers with d=128, 4 attention heads, FFN dimension 512, and dropout 0.1. At each step:

1. Causal self-attention over all previously generated tokens
2. Cross-attention over all GNN node embeddings (the "memory")
3. Action prediction via a linear head
4. Symbol selection via pointer attention: $\text{score}_i = \frac{\mathbf{q}^\top \mathbf{k}_i}{\sqrt{d}}$ where $\mathbf{q}$ is the decoder's query and $\mathbf{k}_i$ is the i-th symbol's projected embedding
5. Variable slot prediction via a separate linear head

KV caching. During inference, we cache self-attention key/value tensors per layer, reducing per-step cost from $O(t^2)$ to $O(t)$ and achieving $\sim$5$\times$ generation speedup.

Algorithm 1 Arity-Constrained Decoding

```
Initialize stack S ← ∅, expecting_pred ← FALSE
for each generation step do
    Compute action logits from decoder
    if expecting_pred then
        Mask all actions except PRED to −∞
    else if S = ∅ then
        Allow only NEW_LIT, END_CLAUSE
    else if S.top.remaining > 0 then
        Allow only ARG_VAR, ARG_FUNC
    else if S.top.remaining = 0 then
        Allow only END_ARGS
    end if
    Sample action from masked logits
    if action = PRED(s) or ARG_FUNC(s) then
        Push arity(s) onto S
    else if action = ARG_VAR then
        S.top.remaining −= 1
    else if action = END_ARGS then
        Pop S; if parent exists, S.top.remaining −= 1
    else if action = NEW_LIT then
        expecting_pred ← TRUE
    end if
end for
```

4.5 Arity-Constrained Decoding

The single most impactful contribution. Without constraints, the decoder frequently generates syntactically invalid clauses (e.g., $\text{eq}(a, b, c)$ for a binary predicate). We introduce a state machine that tracks arity obligations during generation:

This requires **no retraining**—it only modifies the generation procedure. Symbol arities are read from the graph metadata stored during graph construction.

4.6 Alternative Decoder Architectures

We systematically compared five decoders sharing the same GNN encoder:

Plan B: Graph Growing. Predicts literals iteratively: at each step, a cross-attention + self-attention module predicts continue/stop, polarity, predicate (pointer), and arguments. Previously generated literal embeddings provide history via self-attention.

Plan C: Conditional VAE. Wraps the Transformer decoder with a variational autoencoder. A GRU clause encoder maps targets to a posterior $q(\mathbf{z}|\text{clause}, \text{problem})$; a conditional prior $p(\mathbf{z}|\text{problem})$ enables diverse generation via different $\mathbf{z}$ samples. Loss: $\mathcal{L}_{\text{recon}} + 0.1 \cdot D_{\text{KL}}(q\|p)$.

Plan D: SSM (Mamba-style). A selective state-space model with input-dependent A, B, C matrices for selective state updates [5]. Gated linear fusion replaces cross-attention for memory efficiency. With dropout=0.1 added, reaches competitive performance.

Plan E: Subgraph Completion. One-shot prediction: K=6 learnable literal slots attend to encoder outputs via cross-attention and coordinate via self-attention, then each slot predicts active/polarity/predicate/arguments in parallel.

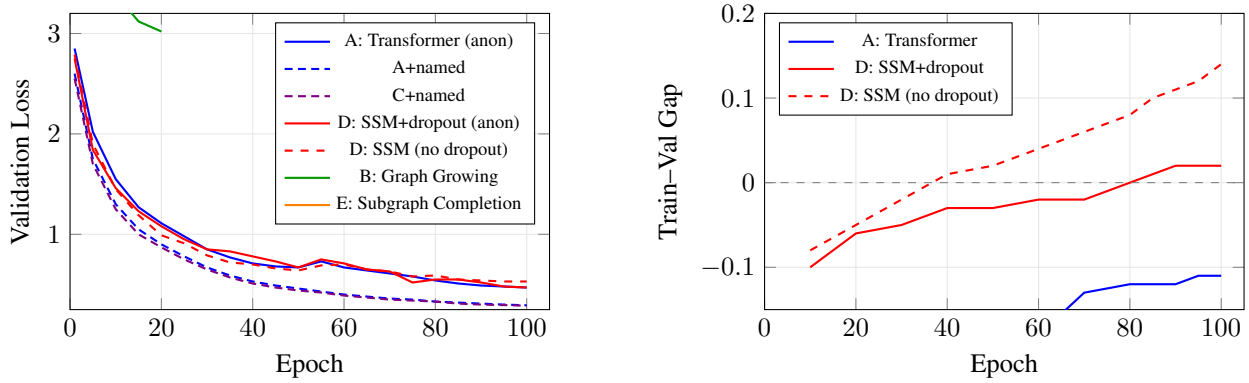

Figure 2: **Left:** Validation loss across training. Named embeddings (dashed blue and violet) reach $\sim$0.29, well below anonymous models ($\sim$0.47). Plans B and E (shown for 20 epochs only) are significantly worse. **Right:** Train–val gap (positive = overfitting). D without dropout overfits progressively; adding dropout eliminates this. A's negative gap is an artifact of dropout being active during training but not evaluation.

Table 3: Decoder architecture comparison. Val loss at 20 epochs (5,000 samples) and 100 epochs (full data, $\sim$44K samples). Training speed on A5000 GPU.

Decoder	**Params**	**Val@20**	**Val@100**	**Train (min)**	**Dropout**
A: Transformer	3.0M	1.71	0.474	48	0.1
D: SSM+dropout	2.6M	1.43	0.473	165	0.1
C: VAE	3.2M	1.75	0.403	$\sim$50	0.1
B: Graph Growing	2.4M	3.02	—	—	0
E: Subgraph Compl.	2.7M	4.90	—	—	0

5 Experiments

5.1 Setup

All models use hidden dimension d=128, 6 GNN layers, and are trained with AdamW (lr=1.2×10^{-3} for batch size 256, weight decay 10^{-5}) with cosine annealing over 100 epochs. Problems with > 1500 graph nodes are filtered (dropping 5.6%). Training uses an NVIDIA A5000 (24GB).

Generation uses top-k (k=10) and nucleus (p=0.9) sampling with temperature varying from 0.7 to 1.0 across attempts, generating 30 conjectures per problem with arity-constrained per-problem batching and `max_steps=80`.

E prover evaluation uses E 2.6 with `-auto` and 10-second timeout. Baselines are computed fresh with the same E binary and parameters.

5.2 Decoder Comparison

Table 3 summarizes the decoder comparison at 20 epochs (all five architectures) and 100 epochs (top configurations including named embeddings).

Key findings. (1) The Transformer and SSM achieve identical final performance when both use dropout. (2) The SSM initially appeared superior (best at 20 epochs) but overfitted without dropout; adding dropout=0.1 closed the gap completely. (3) The VAE (C) achieves 0.403 at 100 epochs, outperforming both A and D in anonymous mode. (4) The Transformer is 3.4$\times$ faster to train and supports KV-cached inference. (5) Graph Growing and Subgraph Completion fail to scale.

Table 4: Impact of named symbol embeddings on validation loss at 100 epochs. Named embeddings add ∼396K parameters for the embedding table plus ∼16K for the expanded input projection.

Model	Anonymous	+Named	Reduction
A: Transformer (100 ep)	0.474	0.293	**38.2%**
C: VAE (100 ep)	0.403	0.289	**28.3%**
D: SSM+dropout (100 ep)	0.473	0.413	**12.7%**

Table 5: Effect of arity-constrained decoding on generation quality. Full-scale evaluation on 2,985 problems, 30 conjectures per problem.

	Syntactic Validity	Both Proved	Useful Speedups	Best Speedup
Without arity constraints	∼2%	∼1.4%	∼31	14.7×
With arity constraints	**85–88%**	**∼61%**	**1,655**	**77×**

5.3 Named Embeddings Impact

Table 4 shows the consistent benefit of named embeddings. The improvement is largest for the Transformer (38%), whose full cross-attention can most effectively exploit the richer symbol representations, and smallest for the SSM (13%), whose gated linear fusion provides a weaker connection to encoder representations. The VAE (C+named) achieves the overall best validation loss of 0.289.

5.4 Arity Constraint Ablation

Table 5 shows the dramatic impact: arity constraints improve syntactic validity from ∼2% to 85–88% (∼43×). The remaining ∼14% of invalid conjectures are almost entirely due to truncation at the `max_steps=80` limit. The useful speedup count increases from ∼31 to 1,655 on the full dataset.

5.5 Test-Set E Prover Evaluation

Table 6 presents the central result: test-set evaluation on 279 problems never seen during training. The best model (D+named) achieves **5.8% useful rate**, generating 161 conjectures that actually speed up the E prover, with best speedup ratios of 0.013–0.015 (67–77×).

Inverse correlation. A striking pattern emerges: models with higher both-proved rates achieve *lower* useful rates (Table 6). A2 and C anon achieve 62.1% both-proved but only 5.1% useful, while D+named achieves 59.4% both-proved but 5.8% useful. This reveals a fundamental exploration–exploitation trade-off: conservative models generate "safe" conjectures that are easily proved but rarely provide speedups, while more adventurous models generate non-obvious lemmas that sometimes fail but yield genuine speedups when they succeed.

5.6 Full-Scale E Prover Evaluation

Table 7 shows the full-scale evaluation confirming the test-set trends. The best configuration (D+named) finds 1,655 useful speedups across all problems, with the best individual speedup ratio of 0.013 (77×) achieved by A+named and C+named.

Best speedups. Table 8 shows the top results. The 77× speedup for `148_tops_1` comes from set-difference decomposition lemmas that the prover's heuristic search fails to discover efficiently. The model generated multiple variants of this insight across different model configurations, all providing dramatic speedups.

Table 6: E prover evaluation on 279 held-out test problems (2,790 conjectures per model). All models use arity-constrained decoding with 10 conjectures per problem. E 2.6 with `-auto` and 10s timeout.

Model	Both Proved	Both %	Useful	Useful %
D+named	1,655	59.4%	**161**	**5.8%**
A+named	1,679	60.0%	159	5.7%
C+named	1,707	60.8%	151	5.4%
A3 (150ep anon)	1,708	60.8%	145	5.2%
C anon	1,742	62.1%	144	5.1%
A2 (100ep anon)	1,744	62.1%	142	5.1%

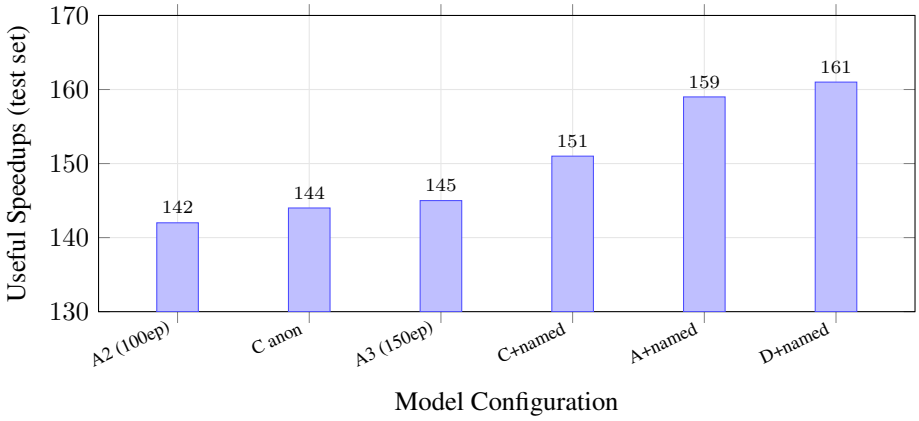

Figure 3: Test-set useful speedups across all six evaluated model configurations (279 problems, 2,790 conjectures each). Named embeddings (right three bars) consistently outperform anonymous models (left three bars). D+named achieves the highest useful count despite having the worst validation loss among named models.

6 Discussion

6.1 Symbol Anonymity as Inductive Bias

Our fully anonymous representation achieves 98% symbol precision, demonstrating that the GNN learns to distinguish symbols by their structural role alone. This is a stronger result than expected: the model not only avoids hallucinating symbols but generates *structurally appropriate* conjectures—e.g., using subset predicates in set-theoretic contexts and cancellation functions in arithmetic contexts.

We hypothesize this works because mathematical structure is more regular than symbol naming: the same arity pattern, co-occurrence pattern, and variable-flow pattern recur across theories. The GNN captures these regularities in 6 rounds of message passing.

6.2 Why Named Embeddings Help

Despite the strong anonymous performance, named embeddings provide a consistent 12–38% validation loss reduction. The improvement comes from several sources:

1. **Disambiguation of structurally similar symbols**: Addition (`k2_xcmplx_0`) and multiplication (`k3_xcmplx_0`) have the same arity and often appear in similar structural positions. Without names, the GNN must rely on subtle higher-order patterns to distinguish them.

2. **Prior knowledge about mathematical semantics**: The name embedding for `m1_subset_1` (subset membership) encodes that this predicate relates to containment, helping the decoder generate appropriate containment-related conjectures.

Table 7: Full E prover evaluation on all 2,985 provable problems (29,850 conjectures per model, 10 per problem).

Model	Both Proved	Both %	Useful	Useful %	Best Ratio
D+named	18,093	61.0%	**1,655**	**5.6%**	0.015
A+named	18,380	61.7%	1,638	5.5%	**0.013**
C+named	18,380	61.7%	1,638	5.5%	**0.013**
D anon	17,921	60.1%	1,506	5.1%	0.014
C anon	∼18,100	∼60.6%	1,408	4.7%	—
A3 (150ep)	∼18,200	∼61%	∼1,350	∼4.5%	—
A2 (100ep)	18,223	61.1%	1,346	4.5%	0.015

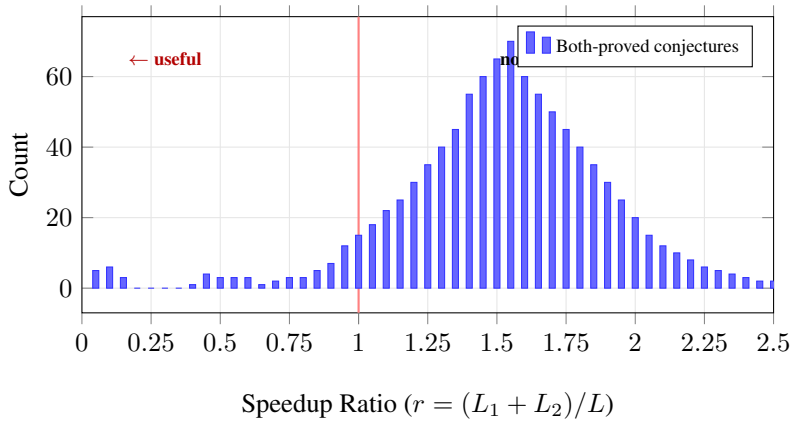

Figure 4: Distribution of speedup ratios for both-proved conjectures. The red line marks r=1.0: conjectures to the left provide actual speedup.

3. **Faster convergence**: Named models reach better validation loss with the same number of epochs because they start with richer symbol representations.

The SSM (D) benefits least from named embeddings (13% reduction vs. 38% for A), likely because its gated linear fusion provides a weaker cross-attention mechanism than the Transformer's full cross-attention, limiting how effectively it can exploit the richer symbol representations. Additionally, D+named shows overfitting (train=0.31 vs. val=0.413), suggesting the additional embedding capacity needs stronger regularization in the SSM architecture.

6.3 Val Loss Does Not Predict Usefulness

Perhaps the most surprising finding: the model with the worst validation loss among named configurations (D+named, 0.413) achieves the best test useful rate (5.8%), while the model with the best validation loss (C+named, 0.289) ranks only third (5.4%).

This disconnect arises because validation loss measures *imitation accuracy* (how well the model reproduces training cuts under teacher forcing), while usefulness measures *generative quality* (how often novel model-generated cuts actually speed up the prover). A model that deviates from training examples (higher loss) but in productive ways (novel cuts that happen to work) can outperform a model that perfectly imitates known cuts.

Practical implication. Validation loss is useful for early stopping and development-time architecture comparison, but final model selection should be based on prover evaluation, not validation loss.

6.4 The Exploration–Exploitation Trade-off

The inverse correlation between both-proved rate and useful rate (Table 6) reveals a fundamental trade-off:

Table 8: Top proof speedups from generated conjectures across all models.

Problem	Model	L	Speedup	Generated Lemma
l48_tops_1	A+named	3889	77×	$x \in A \vee x \notin A \setminus B$
l48_tops_1	C+named	3889	77×	Set difference variant
l48_tops_1	D+named	3889	67×	Set difference variant
l48_tops_1	A2	3889	67×	Set difference variant
l48_tops_1	D anon	3889	71×	Set difference variant
l48_tops_1	various	3889	25–33×	Additional set variants
l21_wsierp_1	various	2455	11.4×	$(x \cdot y)/y = x$ for ordinals
t64_bcialg_1	various	2805	11.2×	BCI-algebra property
l47_jgraph_2	various	3314	7.4×	Graph-theoretic identity

- **Conservative models** (A2, C anon: 62.1% both-proved, 5.1% useful) generate "safe" conjectures that are easily proved but tend to be trivial lemmas providing no search reduction.
- **Adventurous models** (D+named: 59.4% both-proved, 5.8% useful) generate more ambitious conjectures that sometimes fail but provide genuine speedups when they succeed.

The SSM decoder naturally tends toward adventurous generation, possibly because its recurrent state and gated linear fusion create a different internal representation than the Transformer's self-attention, leading to more varied exploration of the conjecture space. The Transformer's dropout, applied in self-attention, cross-attention, and FFN (three places per layer), provides stronger regularization that may over-constrain generation diversity.

6.5 Why 61% of Conjectures Are Valid Lemmas

The high provability rate (both P1 and P2 proved) is remarkable given that the model generates *novel* clauses not seen during training. We attribute this to the pointer mechanism: by selecting only symbols present in the problem, the model is constrained to the right "vocabulary." Combined with arity constraints ensuring correct syntax, the generated clauses are structurally well-formed and use the right symbols—two necessary conditions for provability.

6.6 Limitations

Literal ordering. CNF clauses are unordered disjunctions, but our sequential encoding imposes an arbitrary order. The loss penalizes generating correct literals in the "wrong" order, wasting model capacity.

Batch generation trade-off. Arity constraints require per-problem generation (same-graph batching), which is $\sim 5\times$ slower than cross-problem batching.

Val loss as proxy. Validation loss does not predict prover usefulness well. Proper model selection requires expensive prover evaluation ($\sim$2 hours per model on the full dataset).

Single-clause output. Some useful cuts require multiple cooperating clauses. Our architecture generates one clause at a time.

7 Conclusion

We demonstrated that a GNN encoder with pointer-based autoregressive decoders can generate useful intermediate lemmas for automated theorem proving. The key findings are:

1. **Named symbol embeddings** for $\sim$3,097 Mizar library symbols (with Skolem symbols remaining anonymous) reduce validation loss by 12–38% across all decoder architectures, with the best configuration (C+named) achieving 0.289.

2. **Arity-constrained decoding** transforms validity from 2% to 85–88% with no retraining, a 43× improvement from a simple state machine.

3. **Test-set generalization**: On 279 held-out problems, the best model (D+named) generates 161 useful speedups (5.8% useful rate) with best speedup of **77**×, demonstrating genuine transfer to unseen mathematical theories.

4. **Val loss ≠ usefulness**: The model with worst validation loss among named configurations achieves the best test useful rate, revealing an exploration–exploitation trade-off fundamental to conjecture generation.

5. **Ensemble potential**: Different architectures find different useful cuts (A+named: 159, D+named: 161, not fully overlapping), suggesting that combining models could substantially increase useful cut discovery.

The model learns genuine mathematical insights—set decomposition lemmas, arithmetic cancellation laws, algebraic properties—purely from the graph representation, generating lemmas that provide up to 77× proof speedup as measured by the E prover. This work opens several directions: training with prover feedback (RL), grammar-constrained decoding for guaranteed validity, multi-clause cut generation, ensemble strategies, and scaling to larger mathematical corpora.

References

[1] A. A. Alemi, F. Chollet, N. Een, G. Irving, C. Szegedy, and J. Urban. DeepMath – Deep Sequence Models for Premise Selection. In *NeurIPS*, 2016.

[2] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. HOList: An Environment for Machine Learning of Higher Order Logic Theorem Proving. In *ICML*, 2019.

[3] M. Crouse, I. Abdelaziz, B. Makni, S. Whitehead, C. Cornelio, P. Kapanipathi, K. Srinivas, V. Thost, M. Witbrock, and A. Fokoue. Improving Graph Neural Network Representations of Logical Formulae with Subgraph Pooling. *arXiv:1911.06904*, 2019.

[4] T. Gauthier, C. Kaliszyk, J. Urban, R. Kumar, and M. Norrish. TacticToe: Learning to Reason with HOL4 Tactics. In *LPAR*, 2017.

[5] A. Gu and T. Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*, 2023.

[6] W. Hamilton, Z. Ying, and J. Leskovec. Inductive Representation Learning on Large Graphs. In *NeurIPS*, 2017.

[7] J. Jakubův and J. Urban. ENIGMA: Efficient Learning-Based Inference Guiding Machine. In *CICM*, pages 292–302, 2017.

[8] C. Kaliszyk and J. Urban. MizAR 40 for Mizar 40. *J. Autom. Reasoning*, 55(3):245–256, 2015.

[9] L. Kovács and A. Voronkov. First-Order Theorem Proving and Vampire. In *CAV*, 2013.

[10] S. Loos, G. Irving, C. Szegedy, and C. Kaliszyk. Deep Network Guided Proof Search. In *LPAR*, 2017.

[11] A. Paliwal, S. Loos, M. Rabe, K. Bansal, and C. Szegedy. Graph Representations for Higher-Order Logic and Theorem Proving. In *AAAI*, 2020.

[12] S. Schulz. E – A Brainiac Theorem Prover. *AI Communications*, 15(2-3):111–126, 2002.

[13] D. Selsam, M. Lamm, B. Bunz, P. Liang, L. de Moura, and D. L. Dill. Learning a SAT Solver from Single-Bit Supervision. In *ICLR*, 2019.

[14] O. Vinyals, M. Fortunato, and N. Jaitly. Pointer Networks. In *NeurIPS*, 2015.

[15] M. Wang, Y. Tang, J. Wang, and J. Deng. Premise Selection for Theorem Proving by Deep Graph Embedding. In *NeurIPS*, 2017.

[16] C. Weidenbach, D. Dimova, A. Fietzke, R. Kumar, M. Suda, and P. Wischnewski. SPASS Version 3.5. In *CADE*, 2009.

[17] M. Wu, M. Norrish, C. Walder, and A. Dezfouli. INT: An Inequality Benchmark for Evaluating Generalization in Theorem Proving. In *ICLR*, 2021.

A Detailed Training Curves

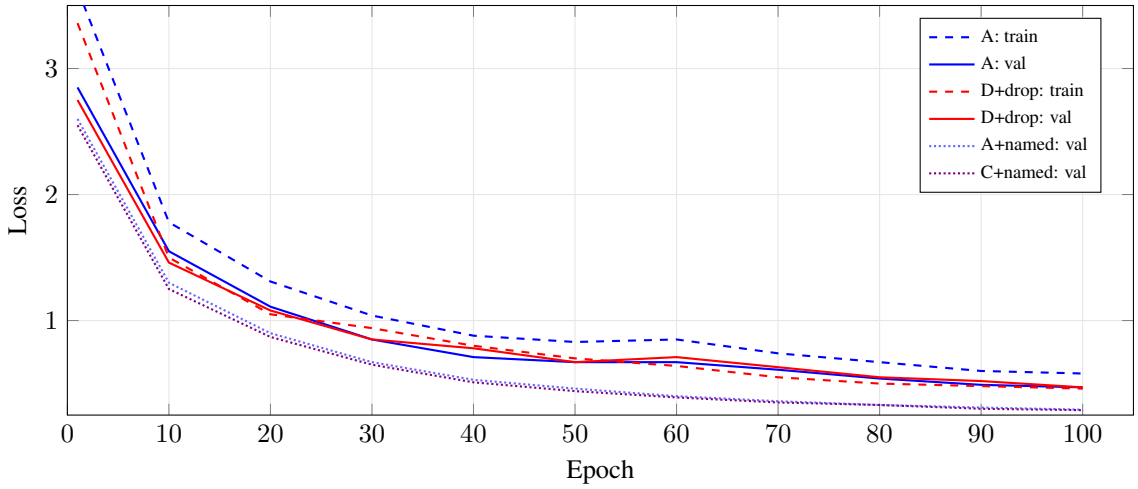

Figure 5: Full training curves for all competitive configurations. Anonymous models (A, D+dropout) converge to val≈0.47. Named models (A+named, C+named) converge to val≈0.29, a 38% reduction. A's train loss is higher than val due to dropout being active during training but not evaluation.

B All Validation Loss Results

Table 9: Complete validation loss results across all configurations.

Model	Mode	Val Loss	Notes
C (VAE)	+named, 100ep	**0.289**	Best overall
A (Transformer)	+named, 100ep	0.293	
A (Transformer)	anonymous, 150ep	0.403	Extended training
C (VAE)	anonymous, 100ep	0.403	
D (SSM+dropout)	+named, 100ep	0.413	Overfitting (train=0.31)
D (SSM+dropout)	anonymous, 100ep	0.473	
A (Transformer)	anonymous, 100ep	0.474	
D (SSM, no dropout)	anonymous, 100ep	0.527	Severe overfitting
B (Graph Growing)	anonymous, 20ep	3.02	Failed to scale
E (Subgraph Compl.)	anonymous, 20ep	4.90	Failed to scale

C Sample Generated Conjectures

l48_tops_1 (topological spaces, L=3889): Five generated conjectures providing $> 12\times$ speedup, all capturing set-difference decomposition:

1. `r2_hidden(X1,X2) | ~r2_hidden(X1,k6_subset_1(X2,X3))` — r=0.013 (77×, A+named)
2. `r2_hidden(X1,k6_subset_1(X2,X3)) | r2_hidden(X1,X3) | ~r2_hidden(X1,X2)` — r=0.034 (29.2×)
3. `r2_hidden(esk4_2(k6_subset_1(X1,X2),X3),X1) | r1_tarski(k6_subset_1(X1,X2),X3) | ~r2_hidden(X3,esk1_0)` — r=0.040 (25.3×)

l21_wsierp_1 (Sierpinski numbers, L=2455): Arithmetic cancellation lemmas:

1. `$eq(k7_xcmplx_0(k3_xcmplx_0(X1,X2),X2),X1) | ~v7_ordinal1(X1)` — r=0.088 (11.4×)
2. `$eq(k7_xcmplx_0(k3_xcmplx_0(k6_numbers,X1),X1),k6_numbers) | ~v7_ordinal1(X1)` — r=0.101 (9.9×)

l100_fomodel0 (formal models, L=296): All 10 tested conjectures were both P1 and P2 proved (100% valid lemma rate), though none provided speedup on this easy problem. Example:

1. `$eq(k17_fomodel0(esk3_0,X1,k9_finseq_1(esk4_0)), k9_finseq_1(esk2_0))` — ratio 1.54 (valid but not useful)

D Arity Constraint State Machine

The arity constraint maintains a stack of outstanding argument obligations. After selecting predicate p with arity n, exactly n arguments must be provided before END_ARGS is allowed. Nested function applications push additional obligations onto the stack.

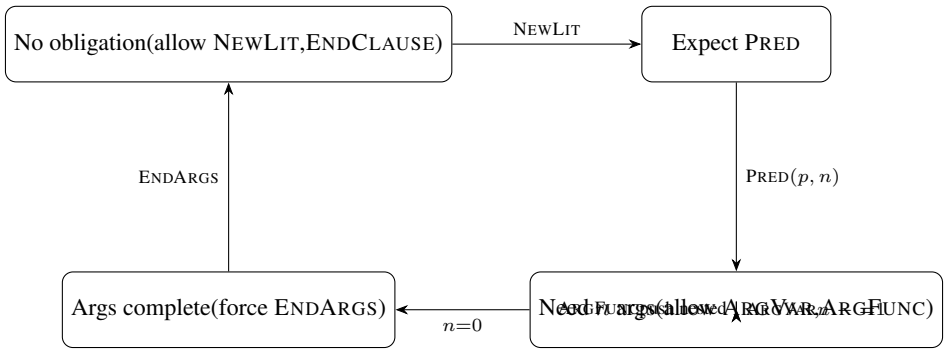

Figure 6: State machine for arity-constrained decoding. Transitions show which actions are allowed in each state. The stack handles nested function applications.

E Generation Validity Rates by Model

Table 10: Syntactic validity with arity-constrained decoding (per-problem mode, `max_steps=80`). Remaining invalidity is almost entirely truncation.

Model	Validity Rate
C anonymous	87.7%
A+named	86.2%
C+named	85.0%
D+named	83.1%
D+dropout anonymous	80.8%
Any model, no constraints	∼2%